TP 21 : Calcul des séquents

1 Préliminaires

Dans ce sujet, nous nous intéressons aux formules du calcul propositionnel. Nous supposons fixé un ensemble $\mathscr{P}$ dénombrable de variables propositionnelles. Nous considérons l'ensemble $\mathscr{F}_0$ des formules défini inductivement à partir des variables propositionnelles et à l'aide des connecteurs $\neg$ (négation), $\vee$ (disjonction), $\wedge$ (conjonction) et $\rightarrow$ (implication). Nous incluons la formule $\bot$ dans cette définition inductive, qui est rappelée en Figure 1. La formule $\top$ n'est pas introduite car n'est que rarement utile.

$$\frac{p \in \mathscr{P}}{p \in \mathscr{F}_0} \qquad \frac{}{\bot \in \mathscr{F}_0} \qquad \frac{A \in \mathscr{F}_0}{\neg A \in \mathscr{F}_0}$$

$$\frac{A \in \mathscr{F}_0 \quad B \in \mathscr{F}_0}{A \wedge B \in \mathscr{F}_0} \qquad \frac{A \in \mathscr{F}_0 \quad B \in \mathscr{F}_0}{A \vee B \in \mathscr{F}_0} \qquad \frac{A \in \mathscr{F}_0 \quad B \in \mathscr{F}_0}{A \rightarrow B \in \mathscr{F}_0}$$

FIGURE 1 – Définition inductive des formules du calcul propositionnel

Pour toute famille finie de formules $(A_i)_{i \in [\![0,n]\!]}$ et tout entier $k \in [\![0,n]\!]$, nous nous permettrons d'écrire $\bigwedge_{i=0}^{k} A_i$ pour désigner la formule définie par :

$$\bigwedge_{i=0}^{k} A_i = \begin{cases} A_0 & \text{si } k = 0 \\ \left(\bigwedge_{i=0}^{k-1} A_i\right) \wedge A_k & \text{sinon} \end{cases}.$$

Une définition similaire s'appliquera également au connecteur $\vee$, et si Γ est un ensemble fini de formules, nous nous permettrons d'écrire $\bigvee_{A \in \Gamma} A$ pour désigner $\bigvee_{i=0}^{n} A_i$, où $A_0, \ldots, A_n$ est une numérotation arbitraire des éléments de Γ. Par convention, nous définirons $\bigvee_{A \in \emptyset} A = \bot$.

Nous voulons confier la recherche de preuve à un ordinateur. Cependant, le système déductif de la logique classique présente plusieurs freins à cet usage :

- Certaines règles, comme $\vee_e$ et $\rightarrow_e$, nécessitent de trouver une formule qui n'apparaît pas nécessairement dans le contexte initial. Trouver une telle formule de manière automatique ne paraît pas aisé.
- Il est possible de « se tromper » en construisant un arbre de preuve, par exemple si l'on applique la règle $\vee_{i,g}$ alors que c'était le membre droit de la disjonction qui était démontrable.

Pour résoudre ces problèmes techniques de la déduction naturelle, Gerhard Gentzen a introduit un système déductif appelé calcul des séquents et qui s'appuie sur la notion de séquent symétrique. Un séquent symétrique est un séquent dont les deux membres sont des ensembles de formules. Nous écrirons donc un séquent symétrique sous la forme $\Gamma \vdash \Delta$ où Γ et Δ sont tous deux des ensembles de formules, contrairement aux séquents asymétriques, de la forme $\Gamma \vdash A$ avec A une formule unique. Le séquent symétrique $\Gamma \vdash \Delta$ doit être interprété de la manière suivante : de la conjonction des formules de Γ, on peut déduire la disjonction des formules de Δ. Les règles du calcul des séquents classique sont présentées en Figure 2.

Bien que le système déductif de la Figure 2 ne contienne aucune règle telle que la règle RAA de la déduction naturelle pour le raisonnement par l'absurde, c'est bien un système pour la logique classique, comme le démontre la question suivante.

Question 1 : Donner une dérivation du tiers exclu : $\vdash A \vee \neg A$. Faire de même avec le séquent $\vdash \neg\neg A \longleftrightarrow A$.

Nous admettons que le calcul des séquents est équivalent à la déduction naturelle, c'est-à-dire que :

- un séquent $\Gamma \vdash A$ est dérivable en déduction naturelle si et seulement s'il l'est également en calcul des séquents ;
- un séquent $\Gamma \vdash \Delta$ est dérivable en calcul des séquents si et seulement si le séquent $\Gamma \vdash \bigvee_{A \in \Delta} A$ l'est également en déduction naturelle.

$$\frac{}{\Gamma, A \vdash A, \Delta}(ax) \qquad \frac{}{\Gamma, \bot \vdash \Delta}(\bot_g)$$

$$\frac{\Gamma \vdash A, \Delta}{\Gamma, \neg A \vdash \Delta}(\neg_g) \qquad \frac{\Gamma, A \vdash \Delta}{\Gamma \vdash \Delta, \neg A}(\neg_d)$$

$$\frac{\Gamma, A, B \vdash \Delta}{\Gamma, A \wedge B \vdash \Delta}(\wedge_g) \qquad \frac{\Gamma \vdash A, \Delta \quad \Gamma, \vdash B, \Delta}{\Gamma \vdash A \wedge B, \Delta}(\wedge_d)$$

$$\frac{\Gamma, A \vdash \Delta \quad \Gamma, B \vdash \Delta}{\Gamma, A \vee B \vdash \Delta}(\vee_g) \qquad \frac{\Gamma \vdash A, B, \Delta}{\Gamma \vdash A \vee B, \Delta}(\vee_d)$$

$$\frac{\Gamma \vdash A, \Delta \quad \Gamma, B \vdash \Delta}{\Gamma, A \rightarrow B \vdash \Delta}(\rightarrow_g) \qquad \frac{\Gamma, A \vdash B, \Delta}{\Gamma \vdash A \rightarrow B, \Delta}(\rightarrow_d)$$

FIGURE 2 – Règles d'inférence de la déduction naturelle classique

1.1 Manipulation des formules

Pour simplifier notre étude du calcul des séquents, nous nous limiterons aux formules n'utilisant que le $\wedge$ et le $\neg$, ce qui nous permettra de nous passer des règles pour la disjonction et l'implication ($\vee$g, $\vee$d, $\rightarrow$g et $\rightarrow$d).

Nous étendons la notion de séquent valide aux séquents symétriques de la manière suivante : $\Gamma \vdash \Delta$ est valide si et seulement si $\Gamma \models \bigvee_{A \in \Delta} A$, ce que nous noterons également $\Gamma \models \Delta$. Nous admettons que le calcul des séquents est un système déductif correct et complet, c'est-à-dire qu'un séquent $\Gamma \vdash \Delta$ est dérivable si et seulement s'il est valide.

Nous souhaitons implémenter un algorithme de recherche de preuve en calcul des séquents. Pour ce faire, nous commençons par nous donner un type pour les formules.

```
type 'a formule =
    | Faux
    | Var of 'a
    | Non of 'a formule
    | Et of 'a formule * 'a formule
```

Nous programmons tout d'abord une fonction utilitaire.

Question 2 : Écrire une fonction `est_atomique : 'a formule -> bool` qui détermine si une formule est atomique, c'est-à-dire si elle ne contient aucun connecteur $\neg$ ou $\wedge$.

Question 3 : Implémenter une fonction `taille : 'a formule -> int` qui calcule la taille d'une formule, définie comme le nombre de connecteurs (négation et conjonction) qu'elle contient.

Question 4 : Écrire une fonction `hauteur : 'a formule -> int` qui détermine la hauteur d'une formule, définie comme la profondeur maximale de l'arbre syntaxique correspondant.

1.2 Vérification et recherche de preuves

Nous nous donnons un type représentant les séquents : nous utilisons un enregistrement contenant une liste `hyp` de formules qui constituent les hypothèses du séquent et une liste `concl` de formules qui constituent sa conclusion.

```
type 'a sequent = { hyp : 'a formule list ; concl : 'a formule list }
```

Nous rappelons que la construction d'un élément de ce type dont les champs valent `l1` et `l2` se fait à l'aide de la syntaxe `{hyp = l1; concl = l2}` et que si `s` est un élément de ce type, on peut lire ses champs à l'aide des expressions `s.hyp` et `s.concl`.

Question 5 : Écrire une fonction `extrait : 'a -> 'a list -> 'a list` telle que `extrait x l` supprime la première occurrence de `x` dans `l` s'il en existe une et lève l'exception `Not_found` sinon.

Question 6 : Écrire une fonction `element_commun : 'a list -> 'a list -> 'a option` qui renvoie un élément commun aux deux listes passées en argument, sous la forme `Some x`, s'il en existe un, et `None` sinon.

Nous appelons formule principale d'une règle d'inférence la formule située dans la conclusion de cette règle et qui en permet l'instanciation. Le tableau suivant indique pour chaque règle de la figure 2 quelle est sa formule principale.

Règle	Formule principale
ax	A(à gauche et à droite de $\vdash$)
$\bot_g$	$\bot$
$\neg_g, \neg_d$	$\neg A$
$\wedge_g, \wedge_d$	$A \wedge B$
$\vee_g, \vee_d$	$A \vee B$
$\rightarrow_g, \rightarrow_d$	$A \rightarrow B$

Nous représentons enfin une dérivation de la manière suivante : nous utilisons un type somme dont chaque constructeur correspond à une règle d'inférence. Chaque constructeur est paramétré par la formule principale de la règle d'inférence, ainsi que par des dérivations de ses prémisses. Nous faisons une exception pour le constructeur `Faux_g`, car l'unique formule permettant d'instancier la règle $\bot$g est $\bot$.

```
type 'a derivation =
    | Axiome of 'a formule
    | Faux_g
    | Non_g of 'a formule * 'a derivation
    | Non_d of 'a formule * 'a derivation
    | Et_g of 'a formule * 'a derivation
    | Et_d of 'a formule * 'a derivation * 'a derivation
```

Par exemple, l'expression `Faux_g` représente n'importe quelle dérivation de la forme $\dfrac{}{\Gamma, \bot \vdash \Delta}\ \bot_g$ où Γ et Δ sont des ensembles de formules quelconques, tandis que l'expression

```
Et_d ( Et ( Var "p", Var "q" ),
       Axiome ( Var "p" ),
       Non_d ( Non ( Var "q" ), Axiome ( Var "q" ) ) )
```

représente les dérivations de la forme $\dfrac{\dfrac{}{\Gamma, p \vdash p, \neg p, \Delta}\ ax \qquad \dfrac{\dfrac{}{\Gamma, p, q \vdash q, \Delta}\ ax}{\Gamma, p \vdash q, \neg q, \Delta}\ \neg_d}{\Gamma, p \vdash p \wedge q, \neg q, \Delta}\ \wedge_d$

Une expression du type `'a derivation` peut représenter plusieurs dérivations différentes, puisqu'il peut exister dans le séquent dérivé des formules dont une telle expression ne fait pas mention (celles des ensembles Γ et Δ dans les exemples ci-avant). Nous nous proposons alors de vérifier si un séquent donné admet bien une dérivation représentée par une telle expression.

Question 7 : Écrire une fonction `est_valide : 'a derivation -> 'a sequent -> bool` qui détermine si une dérivation est un arbre de preuve valide pour un séquent donné. Prenez votre temps pour bien décomposer les différents cas, quitte à sortir un papier et un crayon.

Question 8 : Écrire une fonction `est_trivial : 'a sequent -> bool` qui vérifie si un séquent est trivialement valide, c'est-à-dire s'il contient une même formule dans ses hypothèses et dans ses conclusions.

Question 9 : Écrire une fonction `est_derivable : 'a sequent -> bool` qui tente de déterminer si un séquent est dérivable en appliquant systématiquement les règles du calcul des séquents jusqu'à éventuellement obtenir un séquent trivial.

Question 10 : Écrire une fonction `trace_derivation : 'a sequent -> 'a derivation option` qui essaie de produire une dérivation d'un séquent donné, en renvoyant `None` si aucune dérivation n'est trouvée.